

82514 · MECATRÓNICA Y ROBÓTICA

Bloque 7

Software robótico: ROS 2, simulación y planificación

Sesiones S32-S37 · 8 horas

25, 26 y 27 de noviembre y 2, 3 y 4 de diciembre de 2026 · IQS Universitat Ramon Llull

Hoja de ruta del bloque

1

S32 • mié 25 nov • 1 h , Arquitectura de ROS 2: nodos, topics, DDS y QoS

2

S33 • jue 26 nov • 1 h , Servicios, acciones, parámetros, launch y colcon

3

S34 • vie 27 nov • 2 h , Taller: nodo rclpy, simulación en Gazebo y TF2

4

S35 • mié 2 dic • 1 h , Planificación: mapas, A*, PRM y RRT

5

S36 • jue 3 dic • 1 h , Navegación autónoma con Nav2

6

S37 • vie 4 dic • 2 h , Manipulación con MoveIt 2 e integración del stack

S32

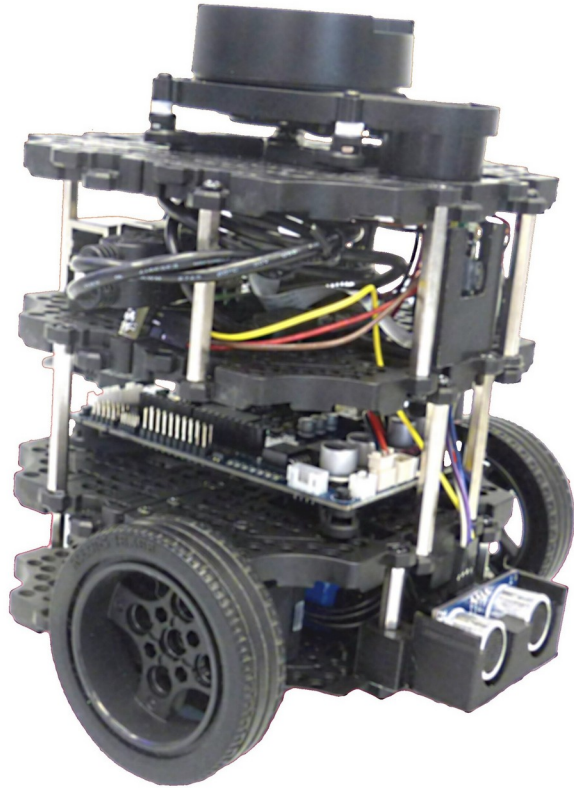
Arquitectura de ROS 2

Grafo de computación · topics y mensajes · DDS y QoS



Roomba: un grafo de sensores y nodos con ruedas

Las plataformas del stack



TurtleBot3: la plataforma de referencia de ROS 2 y Nav2



Flota de robots de almacén (Ocado): el AMR de S6, en producción

De un programa único a un grafo de procesos

La idea que organiza todo el ecosistema

- Un sistema robótico no es un programa: es un conjunto de procesos independientes, nodos, que se comunican por mensajes.
- Cada nodo hace una cosa: leer el LiDAR, estimar la pose, planificar, mandar velocidades a las ruedas.
- Ventaja de ingeniería: se sustituye un nodo sin tocar los demás, se reutiliza código entre robots y se depura por partes.
- El precio: ya no hay una traza de ejecución única; depurar exige inspeccionar el grafo y los flujos de mensajes.

Herramientas de inspección

ros2 node list, qué está corriendo
ros2 topic list, qué canales existen
ros2 topic echo, ver mensajes en vivo
ros2 topic hz, frecuencia real
rqt_graph, el grafo dibujado

Estos cinco comandos resuelven la mayoría de los problemas del taller.

DDS y calidad de servicio

DDS: el transporte real

ROS 2 no inventa su capa de red: se apoya en DDS, un estándar industrial de publicación/suscripción con descubrimiento automático. Por eso no hay un nodo maestro central como en ROS 1.

Por qué importa en el aula

Si dos ordenadores comparten red, sus grafos se ven entre sí. Aislar los equipos con ROS_DOMAIN_ID distintos evita horas de confusión en el taller.

1

Fiabilidad

reliable reenvía hasta confirmar; best effort descarta. Un LiDAR a 40 Hz prefiere best effort; una orden de parada, reliable

2

Durabilidad

transient local guarda el último mensaje para quien llegue tarde: así funcionan los mapas y las descripciones del robot

3

Perfil incompatible

Si publicador y suscriptor no acuerdan perfil, no se conectan y no hay error visible: causa clásica de «no llega nada»



S33

Servicios, acciones y orquestación

Servicios · acciones · parámetros · launch · colcon

Tres formas de comunicar, tres semánticas

1 Topic

Flujo continuo, sin respuesta, muchos a muchos.

Datos de sensor, comandos de velocidad, estado.

«Publico lo que sé; quien quiera, que escuche».

2 Servicio

Petición y respuesta, bloqueante y breve.

Consultar una configuración, activar un modo.

«Te pregunto y espero tu respuesta ahora».

3 Acción

Objetivo de larga duración con retroalimentación y cancelación.

Navegar a un punto, ejecutar una trayectoria.

«Encárgate de esto, ve informando, puedo cancelar».

Regla práctica: si la tarea puede tardar y podría querer cancelarse, es una acción. Nav2 y MoveIt 2 exponen precisamente sus interfaces principales como acciones.

Configurar, lanzar y construir

1 Parámetros

Configuración con nombre y tipo por nodo, ajustable sin recompilar y consultable en marcha: el mismo nodo sirve para varios robots

2 Launch files

Un sistema real arranca decenas de nodos; el launch los declara con sus parámetros y remapeos, y los levanta con una orden

3 Workspace y paquetes

El código vive en paquetes dentro de un workspace; el paquete es la unidad de reutilización y distribución

4 colcon

Construye todos los paquetes resolviendo dependencias. Con `--symlink-install` los cambios en Python se aplican sin reconstruir

Antes de S34: `ros2 pkg create --build-type ament_python mi_paquete • colcon build --symlink-install • source install/setup.bash`



S34

Taller: rclpy, Gazebo y TF2

Nodo en Python · simulación · árbol de transformadas

Anatomía de un nodo y del taller

Un nodo rclpy en cinco piezas

- Heredar de Node y darle nombre; crear publicadores y suscriptores con su tipo de mensaje, tema y perfil de QoS.
- Un temporizador marca el ritmo de publicación; las devoluciones de llamada reaccionan a lo que llega.
- rclpy.spin cede el control al ejecutor: a partir de ahí manda el flujo de mensajes, no el programa.
- El punto de entrada se declara en setup.py para que colcon genere el ejecutable.

Comandos del taller

```
ros2 run mi_paquete mi_nodo  
ros2 topic echo /cmd_vel  
ros2 topic hz /scan  
rqt_graph
```

```
ros2 launch nav2_bringup  
tb3_simulation_launch.py
```

```
ros2 run tf2_tools view_frames  
ros2 run tf2_ros tf2_echo map base_link
```

Simulación y el árbol de transformadas

Gazebo y el puente ros_gz

El simulador aporta física, sensores sintéticos y mundo; el puente traduce entre los mensajes de Gazebo y los de ROS 2.

Permite desarrollar el stack completo sin tocar hardware, y sin poder romperlo.

TF2: quién está dónde

Mantiene el árbol de transformadas entre todos los marcos del robot y del mundo, con su historia temporal.

Permite preguntar «dónde estaba el sensor respecto al mapa en el instante de esta medida»: sin eso, fusionar sensores es imposible.

La convención que ordena la navegación (REP 105)

map → odom → base_link → marcos de sensores. La transformada odom→base_link es continua pero deriva; la map→odom la corrige a saltos cuando la localización se actualiza. Entender esta cadena explica por qué el robot «salta» en RViz al cerrar un bucle, y conecta directamente con la localización del bloque 6 y con AMCL en S36.



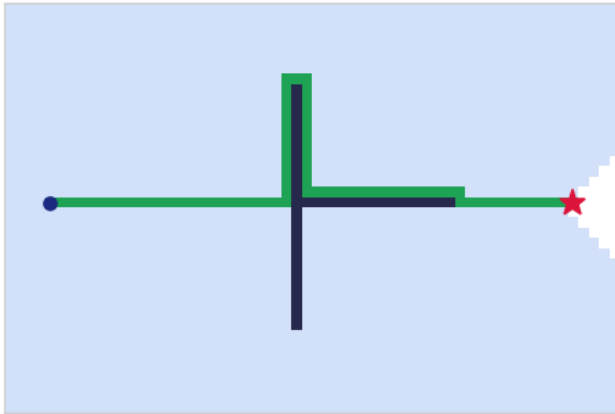
S35

Planificación de trayectorias

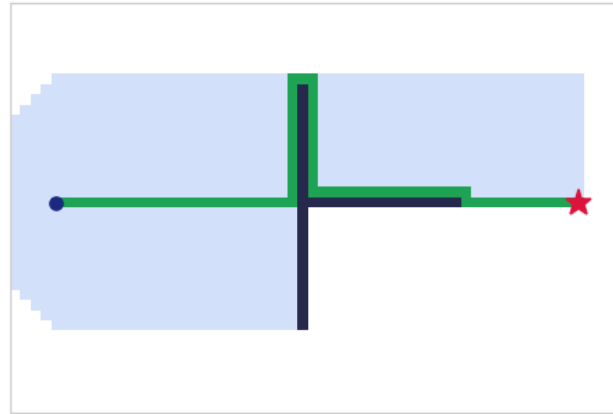
Mapas y C-space · A* · PRM y RRT · campos potenciales

Buscar y muestrear, en imágenes

Dijkstra — 2332 nodos expandidos

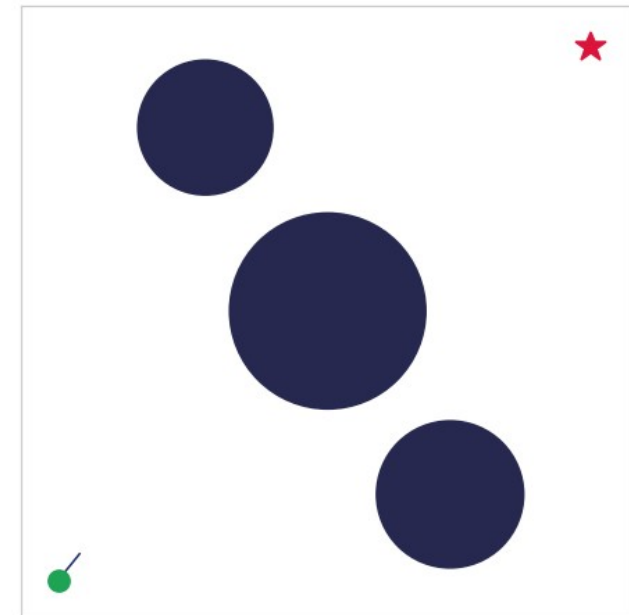


A* (heurística admisible) — 1017 nodos expandidos



Mismo mapa, mismo camino óptimo: A* expande menos de la mitad de nodos

RRT: las muestras tiran del árbol



RRT: las muestras tiran del árbol (cuaderno S35)

Del mapa al espacio de configuraciones

Planificar no es dibujar una línea en el mapa

El problema se plantea en el espacio de configuraciones: hallar un camino en C_{libre} desde la configuración inicial a la meta (Lynch y Park, 2017, p. 353).

- Un robot circular en el plano se convierte en un punto si se inflan los obstáculos con su radio (Lynch y Park, 2017, p. 359).
- Inflar el obstáculo con el tamaño del robot es exactamente lo que hace la capa de inflado del costmap (Corke, 2023, p. 181).
- Completitud: un planificador puede ser completo, completo en resolución o completo probabilísticamente (Lynch y Park, 2017, p. 356).

Dos mapas, dos papeles

Mapa de ocupación: qué celdas están libres, ocupadas o desconocidas, evidencia acumulada (Thrun et al., 2005, pp. 284-286).

Mapa de coste: cuánto cuesta pasar por cada celda, con obstáculos inflados y gradiente de proximidad (Corke, 2023, p. 182).

El primero describe el mundo; el segundo codifica la política de conducción.

Buscar y muestrear

1

Transformada de distancia

Propaga la distancia a la meta por todo el mapa; después basta descender el gradiente. Cara, pero da el camino óptimo desde cualquier punto (Corke, 2023, pp. 177-181).

2

A*

Búsqueda informada: coste acumulado más heurística de coste restante. Garantiza optimalidad solo si la heurística es admisible, es decir, nunca sobreestima (Corke, 2023, p. 175).

3

PRM

Muestrea configuraciones libres, las conecta en un grafo y luego consulta con A*. Compensa cuando hay muchas consultas sobre el mismo mapa (Lynch y Park, 2017, p. 384).

4

RRT

Crece un árbol hacia muestras aleatorias: «las muestras tiran del árbol» (Lynch y Park, 2017, p. 379). Domina en alta dimensión y admite restricciones no holonómicas (Corke, 2023, pp. 196-199).



S36

Navegación autónoma con Nav2

Servidores · costmaps por capas · behavior trees · AMCL



Nav2: cómo se organiza un stack real

1

Servidores especializados

Planificador global, controlador local, suavizador, comportamientos de recuperación: cada uno es un servidor con su interfaz de acción, sustituible por configuración

2

Costmaps por capas

Estática (el mapa), obstáculos (sensores en vivo), inflado (margen del robot). Se combinan en un único mapa de coste maestro

3

Behavior tree

La lógica de navegación no está cableada: es un árbol que decide qué hacer y cómo reaccionar al fallo, reintentar, limpiar el costmap, girar en el sitio

4

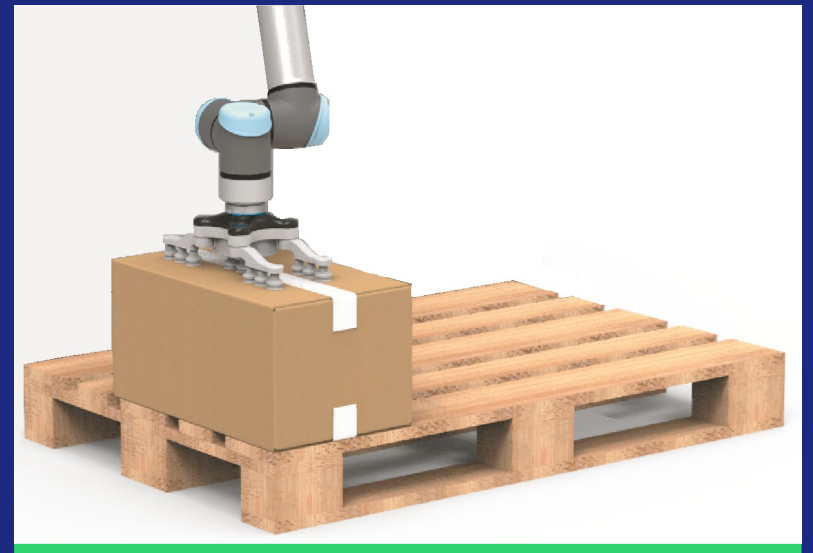
AMCL

La localización de Nav2 es el filtro de partículas adaptativo del bloque 6: aquí es donde aquella teoría se convierte en un nodo que se lanza

S37

Movelt 2 e integración del stack

move_group · planning scene · OMPL · planificación cartesiana



Pick and place: la tarea canónica de MoveIt

Movelt 2: planificar con el mundo dentro

move_group

El nodo orquestador: recibe objetivos como acción, consulta la cinemática del bloque 4, invoca al planificador y devuelve una trayectoria ejecutable.

Planning scene

El modelo del mundo del manipulador: el robot, los obstáculos conocidos y los objetos percibidos.

Planificar sin escena es planificar contra el aire: aquí entra la percepción del bloque 6.

OMPL y la elección del planificador

La biblioteca de planificadores de muestreo por defecto. RRT-Connect, el RRT bidireccional de Lynch y Park, 2017, p. 382, es el caballo de batalla: rápido, sin garantía de optimalidad.

Cartesiano frente a articular

Si la tarea exige una trayectoria recta del efector, soldar, dispensar, insertar, se planifica en el espacio cartesiano y se paga en riesgo de singularidad (bloque 4). Si no, la planificación articular es más robusta.